Homework 3 (100 pts)

Rule: Finish all of the following on your own. Submit your solution in PDF format.

1. **What are the properties of hash functions?**

   The properties of hash functions are ***performance*** (the hash H(m) should be quick and efficient to calculate for any message m, regardless of size, allowing practical use in real-world systems), the ***one-way property*** (given a hash output H(m), it should be computationally infeasible to find the original message m (i.e. hashes are irreversible)), ***weak collision resistance*** (given a message m and its hash H(m), it's hard to find a different message m' where H(m') = H(m)), and ***strong collision resistance*** (it's computationally infeasible to find any two distinct messages m1 and m2 where H(m1) = H(m2), guarding against general collisions (e.g., via the birthday paradox, requiring about 2^{n/2} effort for an n-bit hash)).

2. **Bob designed a new scheme to detect whether a message is modified by an attacker (protecting data integrity). The scheme is to append to each message a SHA-256 hash of that message. Explain the vulnerability of Bob's design.**

   Bob's scheme uses a plain SHA-256 hash appended to the message (e.g., send M || H(M), where H is SHA-256) to detect modifications. While SHA-256 is a strong cryptographic hash (one-way, collision-resistant), this design is vulnerable because it lacks a secret key or authentication mechanism.

   An attacker intercepting M || H(M) can easily modify M to a new message M' (e.g., change a bank transfer amount), then recompute the hash H(M') themselves (since hashing is public and deterministic) and send the forged M' || H(M'). The receiver, recomputing the hash on M', will see it matches and accept the tampered message as valid.

   This fails to provide true integrity/authenticity, as anyone can generate a valid hash for any message. To fix it, use a keyed hash like HMAC-SHA-256 with a shared secret key, making forgeries infeasible without the key (as in Lecture 8 on Message Authentication).

3. **Bob designed another new scheme to detect whether a message is modified by an attacker (protecting data integrity). The scheme is to append to each message a SHA- 256 hash of that message, then encrypt the whole appended message using AES with a known key to the public. Explain the vulnerability of Bob's design.**

   Bob's scheme is a form of Hash-then-Encrypt: Compute H(M) using SHA-256, append it to get M || H(M), then encrypt the whole thing with AES using a publicly known key, sending the ciphertext C = AES_encrypt(M || H(M)). The receiver decrypts C to recover M' || H', recomputes SHA-256 on M', and checks if it matches H' to verify integrity.

   The vulnerability lies in the public AES key: An attacker can decrypt any intercepted C (since the key is known), extract M || H(M), modify M to a forged M' (e.g., alter a critical detail), recompute the valid hash H(M'), and re-encrypt M' || H(M') with the same public key to create a new valid ciphertext C'. The receiver will decrypt C', verify the hash matches (it does, as it's correctly computed for M'), and accept the tampered message as authentic and unchanged. This provides no real integrity or authenticity, as the encryption doesn't bind the hash securely as anyone can forge valid pairs. To fix, use a secret key for encryption/MAC (e.g., HMAC-then-Encrypt) or authenticated modes

like AES-GCM.

4. **A company is selling an artifact worth millions of dollars. Many interested buyers are in the bidding process, which works as follows. Each buyer sends the information of his/her final number (e.g., $1,000,000.00) to the company to show his/her commitment. And the number must be kept confidential from other buyers and even the company before the deadline. After the deadline, all numbers are released to all buyers and the company, then the buyer with the highest number wins. Design a HASH FUNCTION based scheme to secure the bidding process. Describe steps a buyer and/or the company should do.**

   To design a hash function-based scheme for securing the bidding process, buyers and the company can use a commitment protocol leveraging a cryptographic hash like SHA-256. Each buyer first selects their bid amount (e.g., $1,000,000.00) and generates a unique random nonce or salt (a long random string) to ensure unpredictability and prevent rainbow table attacks. The buyer then computes the hash of the concatenated bid and nonce (e.g., H = SHA-256(bid || nonce)) and sends only this hash commitment to the company before the deadline, keeping the actual bid and nonce secret. The company publicly posts all received commitments to prevent later alterations. After the deadline, each buyer reveals their original bid and nonce to the company and all participants. The company (and others) verifies each bid by recomputing the hash and checking if it matches the committed value; if it does, the bid is valid and binding. The highest verified bid wins, ensuring confidentiality during bidding (hashes hide values), integrity (tampering would mismatch hashes), and fairness (commitments are irrevocable and verifiable post-reveal).

5. **Write your <u>OWN</u> code of computing SHA1 and compute the following hashes.**
   1. **OU**
   2. **Computer Science**
   3. **Crypto**

   You can verify the results of your code using: http://www.sha1-online.com/
   Submit your code and screenshots of your results. Notes:
      1. **You cannot just directly call a SHA1 function/algorithm in a program language. You need to code the internals of SHA1.**
      2. **Writing the correct SHA1 code might be challenging. You need to be quite familiar with the bit operations in programming languages like C. You can use SHA1 code available online as reference. But you cannot directly copy the code from Internet since OU plagiarism system can detect it.**

SHA-1.py > ...

```python
def left_rotate(n, b):
    return ((n << b) | (n >> (32 - b))) & 0xFFFFFFFF

def sha1(message):
    # Convert string to bytes if needed
    if isinstance(message, str):
        message = message.encode('utf-8')

    # Step 1: Initialize hash values (from SHA-1 spec)
    h0 = 0x67452301
    h1 = 0xEFCDAB89
    h2 = 0x98BADCFE
    h3 = 0x10325476
    h4 = 0xC3D2E1F0

    # Step 2: Pre-processing (padding)
    original_byte_len = len(message)
    original_bit_len = original_byte_len * 8
    message += b'\x80'                                  # append 1 bit
    while (len(message) % 64) != 56:                    # pad with zeros
        message += b'\x00'
    message += original_bit_len.to_bytes(8, 'big')      # append 64-bit length

    # Step 3: Process each 512-bit (64-byte) chunk
    for i in range(0, len(message), 64):
        chunk = message[i:i+64]
        w = [0] * 80

        # Break chunk into sixteen 32-bit words
        for j in range(16):
            w[j] = int.from_bytes(chunk[j*4:j*4+4], 'big')

        # Extend to 80 words
        for j in range(16, 80):
            w[j] = left_rotate(w[j-3] ^ w[j-8] ^ w[j-14] ^ w[j-16], 1)

        # Initialize working variables
        a, b, c, d, e = h0, h1, h2, h3, h4
```

SHA-1.py > ...

```python
def sha1(message):
        # 80 rounds
        for j in range(80):
            if 0 ≤ j ≤ 19:
                f = (b & c) | ((~b) & d)
                k = 0x5A827999
            elif 20 ≤ j ≤ 39:
                f = b ^ c ^ d
                k = 0x6ED9EBA1
            elif 40 ≤ j ≤ 59:
                f = (b & c) | (b & d) | (c & d)
                k = 0x8F1BBCDC
            else:
                f = b ^ c ^ d
                k = 0xCA62C1D6

            temp = (left_rotate(a, 5) + f + e + k + w[j]) & 0xFFFFFFFF
            e = d
            d = c
            c = left_rotate(b, 30)
            b = a
            a = temp

        # Add to previous hash
        h0 = (h0 + a) & 0xFFFFFFFF
        h1 = (h1 + b) & 0xFFFFFFFF
        h2 = (h2 + c) & 0xFFFFFFFF
        h3 = (h3 + d) & 0xFFFFFFFF
        h4 = (h4 + e) & 0xFFFFFFFF

    # Final hash as 40-character hex string
    return f'{h0:08x}{h1:08x}{h2:08x}{h3:08x}{h4:08x}'

# ═══ Compute the required hashes ═══
print("SHA1('OU')              =", sha1("OU"))
print("SHA1('Computer Science') =", sha1("Computer Science"))
print("SHA1('Crypto')          =", sha1("Crypto"))
```

PROBLEMS OUTPUT TERMINAL PORTS GITLENS AZURE DEBUG CONSOLE

Filter (e.g. text, !excludeText, t... Code

```
[Running] python -u "c:\Users\joyli\Desktop\School\Junior\Spring 26\Comp Sec\SHA-1.py"
SHA1('OU')              = d29a663de815adfb49ac20b6f2994284c409fd2c
SHA1('Computer Science') = 60b8102bd020e99e3357c3d6079d6e1421c5ce57
SHA1('Crypto')          = e849494484ed2e3c1a93babc3e347d2e98ac8604

[Done] exited with code=0 in 0.112 seconds
```

6. Write a computer program to implement the extended Euclid's algorithm, use your code to compute the following. Submit your code and results.
    a. GCD(10012012,2314213)
    b. GCD(28176412,29108188)
    c. GCD(38172,23812188)
    d. The multiplicative inverse of 12091 mod 24123123.
    e. The multiplicative inverse of 28173928 mod 129182771.
    f. GCD(38172302912791823771723321000 2,23812188332813212739187261) for the graduate session only.

euclid.py > ...

```python
def extended_gcd(a, b):
    # Iterative extended Euclidean algorithm
    x, y, u, v = 0, 1, 1, 0
    while a ≠ 0:
        q, r = b // a, b % a
        m, n = x - u * q, y - v * q
        b, a, x, y, u, v = a, r, u, v, m, n
    gcd = b
    return gcd, x, y

def mod_inverse(a, m):
    gcd, x, y = extended_gcd(a, m)
    if gcd ≠ 1:
        return "No inverse exists (gcd ≠ 1)"
    return x % m
```

euclid.py > ...

```python
# Computations
# a. GCD(10012012, 2314213)
gcd_a, _, _ = extended_gcd(10012012, 2314213)
print("a. GCD(10012012, 2314213) =", gcd_a)

# b. GCD(28176412, 29108188)
gcd_b, _, _ = extended_gcd(28176412, 29108188)
print("b. GCD(28176412, 29108188) =", gcd_b)

# c. GCD(38172, 23812188)
gcd_c, _, _ = extended_gcd(38172, 23812188)
print("c. GCD(38172, 23812188) =", gcd_c)

# d. The multiplicative inverse of 12091 mod 24123123
inv_d = mod_inverse(12091, 24123123)
print("d. Inverse of 12091 mod 24123123 =", inv_d)

# e. The multiplicative inverse of 28173928 mod 129182771
inv_e = mod_inverse(28173928, 129182771)
print("e. Inverse of 28173928 mod 129182771 =", inv_e)

# f. GCD(38172302912791823771723321000 2, 23812188332813212739187261)
gcd_f, _, _ = extended_gcd(38172302912791823771723321000 2, 23812188332813212739187261)
print("f. GCD(38172302912791823771723321000 2, 23812188332813212739187261) =", gcd_f)
```

PROBLEMS OUTPUT TERMINAL PORTS GITLENS AZURE DEBUG CONSOLE

```
[Running] python -u "c:\Users\joyli\Desktop\School\Junior\Spring 26\Comp Sec\euclid.py"
a. GCD(10012012, 2314213) = 1
b. GCD(28176412, 29108188) = 4
c. GCD(38172, 23812188) = 12
d. Inverse of 12091 mod 24123123 = 9187576
e. Inverse of 28173928 mod 129182771 = 101412508
f. GCD(38172302912791823771723321000 2, 23812188332813212739187261) = 1

[Done] exited with code=0 in 0.275 seconds
```

7. **Prove that a=n-1 is always a solution to a^2=1 mod n.**

To prove that a = n-1 is always a solution to a^2 = 1 mod n, assume n > 1.
We know n-1 = -1 mod n since subtracting 1 from a multiple of n leaves a remainder of n-1.
Hence, $a^2 = (n-1)^2$ is the same as $(-1)^2$ = 1 mod n.
Alternatively, we could expand directly: $(n-1)^2 = n^2 - 2n + 1$. Mod n so $n^2 = 0$, -2n = 0, and +1 = 1, resulting in $(n-1)^2$ = 1 mod n.

8. **What are the differences between symmetric and asymmetric key crypto.**

The main differences between symmetric and asymmetric key cryptography are:

- Key Usage
  - Symmetric uses a single shared secret key for both encryption and decryption (e.g., AES, DES), while asymmetric uses a pair including a public key for encryption (widely shared) and a private key for decryption (kept secret, e.g., RSA).
- Speed and Efficiency
  - Symmetric is faster and more efficient for large data (constant-time operations), suitable for bulk encryption, while asymmetric is slower (involves complex math like modular exponentiation) and used for small data or key exchange.
- Key Distribution
  - Symmetric requires secure key sharing beforehand (key management challenge), while asymmetric solves this as there is no need to share private keys and public keys can be openly distributed.
- Applications
  - Symmetric for confidentiality in sessions (e.g., data at rest/transit), asymmetric for key exchange (Diffie-Hellman), digital signatures, and non-repudiation (sender can't deny message).
- Security Basis
  - Symmetric relies on key secrecy and computational hardness (e.g., brute-force resistance), asymmetric on hard math problems (e.g., factoring in RSA).

Overall, symmetric is practical for performance but scales poorly for many users; asymmetric enables secure communication without prior secrets but needs hybrid use (e.g., asymmetric for key exchange, symmetric for data).

9. **Explain the feasibility and security of RSA.**

RSA's feasibility stems from its basis in modular arithmetic and number theory. First, we generate two large primes p and q, compute n = p*q (modulus), phi(n) = (p-1)(q-1), then choose public exponent e coprime to phi(n), and private $d = e^{-1}$ mod phi(n).

Encryption: $C = M^e$ mod n

Decryption: M = C^d mod n

This is feasible because key generation uses efficient primality tests (e.g., Miller-Rabin), and modular exponentiation (via square-and-multiply) is fast even for large numbers (1024-4096 bits). Modern hardware handles it quickly for key exchange.

The security relies on the RSA problem: Factoring n into p and q is computationally infeasible for large primes (e.g., 2048-bit n requires ~2^{100} effort, beyond current tech). No efficient algorithm factors large semiprimes (assuming factoring is hard, considering P vs NP). However, weak implementations (poor randomness, small e, side-channel attacks) can

be vulnerable; use padding (OAEP) to resist chosen-ciphertext attacks. Overall, RSA is secure with large keys but slower than symmetric, so often hybrid (e.g., with AES).

10. **Alice and Bob use the Diffie-Hellman to create two keys k1 and k2. In generating k1, Alice selects a random number x and computes SA = g^x mod p. Bob selects a random number y and computes SB = g^y mod p. Alice and Bob exchange SA and SB, and k1 = g^{xy} mod p. In generating k2. Alice chooses another random number x' but Bob still uses the same random number y. Suppose an eavesdropper knows that x and x' differ each other by t (i.e., either x-x' = t or x'-x=t). Given the knowledge of k1 and a pair of plaintext M and ciphertext C encrypted by k2, can the eavesdropper find out k2?**

    Yes, the eavesdropper can find k2 given k1, the relation t (x' = x ± t), and a known plaintext-ciphertext pair (M, C) encrypted with k2. From Diffie-Hellman, k1 = g^{xy} mod p (shared after exchanging SA = g^x mod p, SB = g^y mod p). For k2, Alice uses x', so k2 = $g^{x'y}$ mod p = $g^{(x \pm t)y}$ mod p = (g^{xy} * $g^{\pm ty}$) mod p = (k1 * $(g^y)^{\pm t}$) mod p. Since SB = g^y mod p is public, the eavesdropper computes $(SB)^{\pm t}$ mod p (using modular exponentiation; for -t, use modular inverse if needed). Then k2 = k1 * $(SB)^{\pm t}$ mod p (or divide for -t). With k2, they decrypt C to recover M (assuming known encryption scheme). This exploits the algebraic relation between x and x', breaking secrecy as reuse or predictable randomness is dangerous in DH.

11. **Alice designs a double-RSA cipher. She first generates two secret primes *p* and *q*, and compute *n*=*p**q*, then choose two public encryption exponents e_1 and e_2 that are relatively prime to ø(*n*). So <e_1, e_2, *n*> becomes the public key. She tells people to encrypt message *M* by computing $C_1 = M^{e_1}$ mod *n* and then $C = C_1^{e_2}$ mod *n*, finally sending just *C* to her.**

    a. **Show the decryption process (i.e., how Alice can obtain the plaintext *M* from the final ciphertext *C*).**

        Alice decrypts C by first applying the inverse of e2 (since outer encryption), then e1. With private exponents d1 = $e1^{-1}$ mod phi(n) and d2 = $e2^{-1}$ mod phi(n) (computed via extended Euclid, from Lecture 13), decrypt as: First, C1' = C^{d2} mod n (inverts e2, recovering C1 = M^{e1} mod n), then M = $C1'^{d1}$ mod n (inverts e1). This works because RSA exponents are modular inverses, and operations commute under the modulus.

    b. **Is the double-RSA cipher more secure, less secure, or just as secure as the regular RSA cipher with the same modulus *n* but only one encryption exponent? Why?**

        The double-RSA is just as secure as regular RSA with the same modulus n but exponent e = e1*e2 mod phi(n). Effective strength depends on factoring n, not the exponent size. Larger e doesn't increase security (exponentiation time similar via efficient algos), and if e1/e2 small, partial attacks (e.g., Coppersmith) might apply, but overall equivalent hardness. It adds no extra protection against factoring attacks on n.

c. **Charlie got Alice's instructions confused, and encrypt message *M* for Alice using e_1 and e_2 in the reverse order (i.e., Charlie uses $C_1 = M^{e_2} \bmod n$ then $C = C_1^{e_1} \bmod n$). What would happen when Alice, unaware of Charlie's error, tries to decipher the ciphertext using her usual procedure?**

If Charlie encrypts as $C1 = M^{e2} \bmod n$ then $C = C1^{e1} \bmod n$ (reversed exponents), Alice, unaware, decrypts as usual: $C1' = C^{d2} \bmod n = (M^{e2\ e1})^{d2} \bmod n = M^{e1\ (e2\ d2)} \bmod n = M^{e1 * 1} \bmod n = M^{e1} \bmod n$ (since e2 d2 = 1 mod phi(n)). Then $M' = C1'^{d1} \bmod n = (M^{e1})^{d1} \bmod n = M^1 \bmod n = M$. Surprisingly, she recovers M correctly. Exponents commute in RSA (multiplication order doesn't matter: e1e2 = e2e1), so decryption order doesn't affect the result. It's equivalent to single exponent e1e2 inverted by d1d2 (mod phi(n)).

12. **In this lab exercise, you will work with a BMP image file named *pic_original.bmp* and explore how encryption affects visual data. Your task is to encrypt the image using OpenSSL in both ECB (Electronic Code Book) and CBC (Cipher Block Chaining) modes. For example, you might use commands like *openssl enc -aes-256- ecb -in pic_original.bmp -out pic_ecb.enc* for ECB mode and a similar command with *-aes-256-cbc* for CBC mode.**

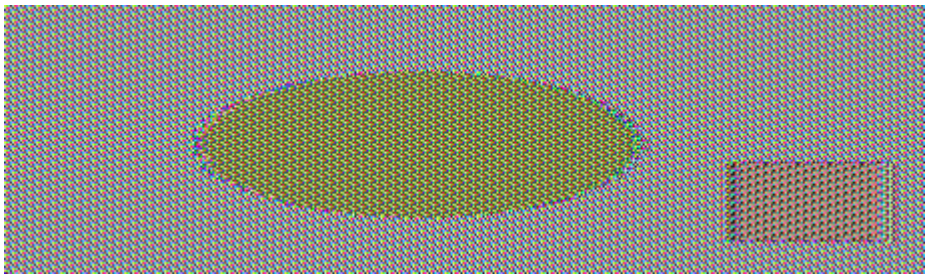
    Because BMP files rely on the first 54 bytes as header information to display correctly, you must extract the header from the original image and combine it with the encrypted data: extract the header with *head -c 54 pic_original.bmp > header*, extract the encrypted body from byte 55 onward using *tail -c +55 pic_ecb.enc > body*, and then concatenate them via *cat header body > pic_ecb.bmp* (repeat for the CBC-encrypted file). Finally, open the resulting BMP files using an image viewer and observe any visible patterns or clues that might reveal aspects of the original picture. Please submit screenshots of your terminal commands, the resulting images, and a brief report discussing your observations and any differences between the ECB and CBC encryption outcomes.

    For Linux and macOS, you can directly use the head and tail commands. For Windows users, you have two options: either run these commands via *WSL* (Windows Subsystem for Linux) or use a hex editor (such as HxD) to manually copy the first 54 bytes from *pic_original.bmp* and overwrite the first 54 bytes in each encrypted file. Then, save the modified files as *pic_ecb.bmp* and *pic_cbc.bmp*.

Commands

```
code-to-joy@JM-SurfacePro1
code-to-joy@JM-SurfacePro11:~/CompSec/ComputerSecurity/assignments/Assig
nment3$ openssl enc -aes-256-ecb -pbkdf2 -salt -in pic_original.bmp -out
 pic_ecb.enc
enter AES-256-ECB encryption password:
Verifying - enter AES-256-ECB encryption password:
code-to-joy@JM-SurfacePro11:~/CompSec/ComputerSecurity/assignments/Assig
nment3$ openssl enc -aes-256-cbc -pbkdf2 -salt -in pic_original.bmp -out
 pic_cbc.enc
enter AES-256-CBC encryption password:
Verifying - enter AES-256-CBC encryption password:
code-to-joy@JM-SurfacePro11:~/CompSec/ComputerSecurity/assignments/Assig
nment3$ head -c 54 pic_original.bmp > header
code-to-joy@JM-SurfacePro11:~/CompSec/ComputerSecurity/assignments/Assig
nment3$ tail -c +55 pic_ecb.enc > body_ecb
code-to-joy@JM-SurfacePro11:~/CompSec/ComputerSecurity/assignments/Assig
nment3$ cat header body_ecb > pic_ecb.bmp
code-to-joy@JM-SurfacePro11:~/CompSec/ComputerSecurity/assignments/Assig
nment3$ tail -c +55 pic_cbc.enc > body_cbc
code-to-joy@JM-SurfacePro11:~/CompSec/ComputerSecurity/assignments/Assig
nment3$ cat header body_cbc > pic_cbc.bmp
code-to-joy@JM-SurfacePro11:~/CompSec/ComputerSecurity/assignments/Assig
nment3$
```

pic_ecb.bmp



pic_cbc.bmp

Observations

Upon viewing, the ECB-encrypted image displays a "shadow" or faint outline of the original shapes including a distorted yellowish oval and orangish square on a dotted rbg background, revealing structural patterns from the plaintext. In contrast, the CBC-encrypted image appears as a completely random, colorful pixel blur resembling static noise, with no discernible shapes or patterns from the original.

These differences stem from the two modes' handling of data: ECB encrypts each block independently, so identical plaintext blocks (common in images with uniform areas like white space) produce identical ciphertext blocks, leaking visual patterns and making it insecure for structured data. CBC, however, chains blocks with an IV and previous ciphertext, providing diffusion where changes propagate, resulting in randomized output that hides the original structure effectively. This highlights why ECB is vulnerable to pattern analysis attacks, while CBC offers better confidentiality for visual media, though both require proper IV management for full security.